

QMS Codebase — Learning Guide

A practical walkthrough of the Quality Management System app: file map, data model, request flow, permissions, and workflow logic.

Contents

- 1. What this app is
- 2. Tech stack & big picture
- 3. Full file map (who does what)
- 4. The data model (Prisma schema)
- 5. The "module" abstraction — key idea
- 6. Auth & session flow
- 7. Permissions & the org hierarchy
- 8. Record lifecycle (status workflow)
- 9. Committees & notification/email
- 10. One request, end to end
- 11. The dashboard & cache
- 12. Running & debugging locally
- 13. Suggested learning path

1. What this app is

A Quality Management System: teams log work records (Documents, CAPAs, Non-conformances, Audits, Training), move them through an approval workflow, manage committees and tasks, and get notified (in-app + email) when something needs their attention.

Key demo accounts (from `prisma/seed.ts`, after `npm run prisma:seed`):

Name	Email	Role	Password
Alicia Reed	alicia.reed@qms.local	Director (monitor-only)	Qms-Director-2026!
Marcus Webb	marcus.webb@qms.local	Deputy Director	Webb-sD6m!39
Mason Lee	mason.lee@qms.local	Quality Lead	Qms-Quality-K5t9!
Priya Shah	priya.shah@qms.local	Operations Lead	QmsOps-R2z7#2026

Noah Foster	noah.foster@qms.local	Process Owner	Foster-x9vE!44
Chris Allen	chris.allen@qms.local	Quality Analyst	Allen-cl4p!12

2. Tech stack & big picture

- **Next.js 16** App Router (Turbopack) — React 19, TypeScript strict
- **Prisma 7 + Neon PostgreSQL** — ORM + hosted DB (driver adapter `@prisma/adapters-pg`)
- **NextAuth.js v4** — Credentials provider, JWT sessions
- **Resend** — transactional email
- **Tailwind v4** — styling

Browser → proxy.ts (auth gate, redirects anon users) → app/ pages (React UI) → app/api/ route handlers → src/lib/ business logic → Prisma → Neon DB └ email.ts → Resend (notification email)

Two module folders exist by design: `app/` is the Next.js App Router (pages + APIs); `src/lib/` holds the pure business logic that pages and API routes share.

3. Full file map (who does what)

3.1 Entry points & app shell

File	Responsibility
<code>app/layout.tsx</code>	Root layout, <code>SessionProvider</code> via <code>app/providers.tsx</code> , fonts, metadata
<code>app/page.tsx</code>	Main app: single client component that hosts dashboard, list, new, detail, edit, approval panel — with client-side navigation (no reloads)
<code>app/module-routes.tsx</code>	Server-side wrapper for <code>/[module]</code> routes — resolves the module key, loads session + profiles, enforces permission gates before rendering <code><Home /></code>
<code>app/dashboard/page.tsx</code>	Standalone dashboard route

<code>app/login/page.tsx</code> , <code>register</code> , <code>forgot-password</code> , <code>reset-password</code> , <code>unauthorized</code>	Auth pages
<code>app/admin/team/page.tsx</code>	Team admin: structure edits, add user, reassign records, succession, committee permissions
<code>app/committees/page.tsx</code> + <code>committees/[id]/page.tsx</code>	Committee list + detail (members, tasks)
<code>app/profiles/[id]/page.tsx</code>	Public profile page (org data, direct reports, memberships)
<code>app/settings/page.tsx</code> , <code>app/notifications/page.tsx</code>	Account settings, notification list + mark-read
<code>proxy.ts</code>	Next 16 proxy: unauthenticated users → <code>/login</code> ; public paths + <code>/api/*</code> pass (APIs self-guard)

3.2 Shared UI components

File	Responsibility
<code>components/qms.tsx</code>	Module core: <code>ModuleKey</code> , <code>moduleConfig</code> (labels/colors/API path), <code>ModuleIcon</code> , <code>StatusBadge</code> , <code>DataTable</code> , <code>RecordForm</code> — all modules reuse these
<code>components/dashboard.tsx</code>	Dashboard tabs: KPIs, <code>ApprovalWidget</code> , <code>TeamRollups</code> , <code>PersonProgress</code> , <code>Compliance</code> — role-aware
<code>components/approval-panel.tsx</code>	Floating Approve/Reject window for the viewer's review queue
<code>components/committee-forms.tsx</code>	<code>AddMember</code> , <code>AssignTask</code> , <code>GrantPermission</code> , <code>Succession</code> forms
<code>components/profile-display.tsx</code>	Reusable profile card (manager, direct reports, team size)

3.3 Business logic (src/lib + lib)

File	Responsibility
<code>src/lib/qms-record-api.ts</code>	The heart. <code>prismaForModule()</code> maps module → Prisma model; <code>list/create/update/delete/transition/comment</code> for all 5 modules; status transitions + history + notifications

<code>src/lib/module-route-factory.ts</code>	Builds identical GET/POST/PATCH/DELETE/submit/approve/revise/comments handlers for every module
<code>src/lib/permissions.ts</code>	Pure permission & org helpers: <code>getDashboardRole</code> , <code>getSubordinateIds</code> , <code>canApproveOrRevise</code> , <code>canCreateRecords</code> , <code>canManageCommittees</code> , monitor-only guards
<code>lib/permissions.ts</code>	Client-side re-export of the above (keeps <code>pg</code> /Prisma out of the browser)
<code>src/lib/api-auth.ts</code>	<code>requireSessionUser()</code> — first call in every API route; returns <code>{ userId, profiles }</code> or a 401 response
<code>lib/auth.ts</code> (+ <code>src/lib/auth.ts</code> , <code>types/next-auth.d.ts</code>)	NextAuth config: Credentials provider, JWT callbacks
<code>src/lib/load-profiles.ts</code>	Server-only profile loader (keeps Prisma import out of client bundles)
<code>src/lib/email.ts</code>	<code>sendEmailNotification()</code> + <code>sendNotification()</code> — combined email + in-app notification
<code>src/lib/succession.ts</code>	Employee succession: re-parent subordinates/records on departure
<code>src/lib/passwords.ts</code>	Password hashing helpers
<code>src/lib/prisma.ts</code> / <code>lib/prisma.ts</code>	Singleton Prisma client with the <code>pg</code> driver adapter
<code>lib/qms-data.ts</code>	Seed profile data (org tree) + module seed rows + cache type
<code>lib/qms-cache.ts</code>	Client-side snapshot cache by <code>viewId</code> (instant back-navigation)
<code>prisma/seed.ts</code>	DB seeding: profiles, module records, committees, tasks, history, notifications; prints credentials

3.4 API routes

A route file's name defines its URL. Groups:

URL	Implements
<code>/api/documents capa nonconformances audits training</code>	Collection: GET list + POST create (via <code>moduleCollectionHandlers</code>)
<code>/api/<module>/[id]</code>	Get / update / soft-delete a record (via <code>moduleItemHandlers</code>)
<code>/api/<module>/[id]/submit approve revise comments</code>	Workflow actions + comments (via <code>moduleActionHandler</code> ,

	<code>moduleCommentHandlers)</code>
<code>/api/records/approve revise assign</code>	Generic approve/reject + assignment (used by the approval panel)
<code>/api/committees , /[id] , /[id]/members , /[id]/tasks</code>	Committee CRUD, membership, task assignment (emails <code>task_assigned</code>)
<code>/api/profiles , /api/profiles/[id]</code>	Profile list (sanitized) + single profile
<code>/api/notifications</code>	GET list + PATCH mark-read
<code>/api/admin/team</code>	POST add user, PATCH role/reportsTo/email + record reassignment, GET all data
<code>/api/admin/permissions , /api/admin/succession , /api/admin/test-email</code>	Top-authority admin operations
<code>/api/account/password</code>	Change own password
<code>/api/auth/[... nextauth]</code>	NextAuth handler

Why " capas " but `/api/capa` ? The user-facing URL uses the plural (" `/capas/1` "), but the API path and module key use the singular. `modulePath()` (`app/page.tsx`) and `resolveModule()` (`app/module-routes.tsx`) convert between them.

4. The data model (Prisma schema)

`prisma/schema.prisma` :

Model	Key fields	Notes
<code>Profile</code>	<code>reportsTo</code> , <code>roleTitle</code> , <code>canManageCommittees</code> , <code>passwordHash</code>	Self-relation = org tree ("ProfileReportsTo"); <code>reportsTo: null</code> = top authority (Director)
<code>Document</code> , <code>Capa</code> , <code>Nonconformance</code> , <code>Audit</code> , <code>Training</code>	<code>assignedTo</code> → <code>Profile</code> , <code>status</code> , <code>deletedAt</code>	Same shape = why one code path serves all five
<code>Committee</code>	<code>name</code> , <code>createdBy</code>	Groups people for collaborative tasks
<code>CommitteeMembership</code>	<code>committeeId</code> , <code>profileId</code> , <code>roleInCommittee</code> ("head"/"member")	Unique per committee+profile

CommitteeTask	committeeId , assignedTo , assignedBy , status	Status: assigned / in_progress / submitted / approved
RecordHistory	recordType , recordId , fromStatus , toStatus , changedBy	Polymorphic-ish audit trail (keyed by type+id)
Comment	recordType , recordId , authorId , body	Discussion on any record
Notification	userId , message , link , read	In-app; every record event creates one
PasswordResetToken	profileId , tokenHash , expiresAt	Forgot-password flow

5. The "module" abstraction — key idea

Five record types behave identically, so they're driven by **one** implementation parameterized by `ModuleKey` :

```

type ModuleKey = "documents" | "capa" | "nonconformances" | "audits" | "training"

// src/lib/qms-record-api.ts
export function prismaForModule(module: ModuleKey): Prisma.DocumentDelegate {
  switch (module) {
    case "documents": return prisma.document as ...;
    case "capa":      return prisma.capa as ...;
    // ... all 5 map to the same-shaped delegate
  }
}

// src/lib/module-route-factory.ts
export function moduleCollectionHandlers(moduleKey: ModuleKey) {
  return {
    GET: () => handleList(moduleKey),
    POST: (request) => handleCreate(moduleKey, request),
  };
}

// app/api/documents/route.ts (each module's whole route file)
import { moduleCollectionHandlers } from "@src/lib/module-route-factory";
export const { GET, POST } = moduleCollectionHandlers("documents");

```

Read it as "documents, capas, nc, audits, training are the same thing wearing different hats." This is the single most important mental model in this codebase.

6. Auth & session flow

GET / → proxy.ts? no session → 307 /login → session ok → route handler →
 getSession(authOptions) (app/module-routes.tsx) → loadProfiles() (all profiles,
 server-side) → pass currentUserId + profile checks → <Home />

API side: every handler starts with `requireSessionUser()` (`src/lib/api-auth.ts`):

```
const auth = await requireSessionUser();
if ("response" in auth) return auth.response; // 401 if not logged in
// auth.userId → the logged-in profile id
// auth.profiles → ALL profiles (for permission math)
```

Login posts to `/api/auth/callback/credentials` (NextAuth): email+password → verified against
`passwordHash` → JWT stores `id`. The client gets the session via `useSession()` and reads `session.user.id`.

7. Permissions & the org hierarchy

Everything derives from the `reportsTo` self-relation: `Director` → `Deputy Director` → `Leads` → `Owners` → `staff`.

Helper	Meaning
<code>getSubordinateIds(userId, profiles)</code>	BFS down the tree from a user (excludes self)
<code>getViewerScope(userId, profiles)</code>	<code>[userId, ... subordinates]</code> — who a user can see/touch
<code>getDashboardRole(userId, profiles)</code>	<code>top-authority</code> <code>team-lead</code> <code>process-owner</code> <code>staff</code>
<code>canViewRecord(viewerId, assignedTo, profiles)</code>	<code>assignedTo</code> in viewer scope
<code>canSubmitOrUpdate(viewerId, assignedTo)</code>	only the assignee
<code>canApproveOrRevise(viewerId, assignedTo, profiles)</code>	walk <code>assignedTo.reportsTo</code> up until you find <code>viewerId</code> (manager chain); also governance: top authority approves everything
<code>isMonitorOnly(userId, profiles)</code>	top authority (Director) — cannot create/assign/manage committees; still approves as final authority
<code>canCreateRecords / canAssignRecords</code>	<code>!isMonitorOnly</code>

<code>canManageCommittees</code>	flag + not monitor-only
<code>canAssignCommitteeTask</code>	manager OR committee head
<code>canGrantCommitteePermission</code> / <code>canInitiateSuccession</code>	top authority (system admin)

Permission check happens **twice**: once server-side in the API route (authoritative), and once in the UI to hide buttons. Never trust the client alone — the API is the guard.

8. Record lifecycle (status workflow)

Draft **--submit-->** Pending **--approve-->** Approved/Closed **--revise-->** back to Draft (with feedback) Also: "In progress", "Done", "Overdue" (training/audit)

The orchestrator is `transitionModuleRecord()` in `src/lib/qms-record-api.ts`. For every transition it:

1. Checks permission by action:

```
submit → canSubmitOrUpdate(userId, record.assignedTo) // assignee only
approve → canApproveOrRevise(userId, record.assignedTo, profiles) // any manager up the chain
revise → canApproveOrRevise(...)
```

2. Updates the record's `status` in the DB.

3. Inserts a `RecordHistory` row (audit trail with comment).

4. Sends a notification + email to the right person:

- **submit** → notifies `record.assigned.reportsTo` (the manager awaiting approval)
- **approve** → notifies the assignee
- **revise** → notifies the assignee with feedback

Because notifications use *dynamic* `reportsTo` lookup, adding the Deputy Director layer immediately rerouted submit-notifications through the deputy without extra code.

9. Committees & notification/email

- **Create committee / add member / assign head** → gated by `canManageCommittees`.
- **Assign committee task** → `canAssignCommitteeTask` (manager OR head); calls `sendNotification({ type: "task_assigned", ... })` which creates the in-app `Notification` and sends email via Resend (`src/lib/email.ts`).

- If `RESEND_API_KEY` is empty, email is skipped but the in-app notification still appears (graceful fallback — `isEmailEnabled()`).
- Notification *recipient address* always comes from `Profile.email` — the Team admin page's Email fields exist so real users can receive real email.

10. One request, end to end

Scenario: Deputy Director lists CAPAs, opens one, and the Quality Lead approves it.

```
1. GET /capas → proxy.ts (auth) → app/[module]/page.tsx → ModuleRoute("capa", "list")
2. app/[module]/page.tsx fetches /api/capa → app/api/capa/route.ts →
  moduleCollectionHandlers("capa").GET → requireSessionUser() → listModuleRecords("capa",
  userId, profiles) → prisma.capa.findMany(where: assignedTo IN viewerScope, deletedAt null)
3. Approve button → POST /api/capa/:id/approve (or /api/records/approve) →
  moduleActionHandler("capa", "approve") → transitionModuleRecord(...) →
  canApproveOrRevise(userId, assignedTo, profiles) ✓ →
  prisma.capa.update(status: "Approved") → RecordHistory insert → sendNotification(approved) →
  Notification row + Resend email
4. Client (app/page.tsx) updates state in place →
  StatusBadge flips to "Approved"
```

11. The dashboard & cache

- `app/page.tsx` is one big client component. Module switching, "New record", edits all use `window.history.replaceState ( syncUrl )` instead of `router.push` — so navigation never remounts the app or refetches everything.
- `lib/qms-cache.ts` : a `localStorage` -backed snapshot (`profiles/records/notifications`) keyed by `viewerId` . On remount the app hydrates instantly from cache then refreshes in the background. Back-navigation is therefore instant.
- `components/dashboard.tsx` : role branching at `Dashboard({ role })` :
 - `top-authority` → Executive Dashboard: company-wide KPIs, org breakdown, compliance score
 - `team-lead` → team subtree KPIs + pending approvals + rollups
 - `process-owner` → direct team KPIs + approval widget + overdue
 - `staff` → "My tasks" + training status

12. Running & debugging locally

Command	What it does
<code>npm run dev</code>	Dev server (hot reload)
<code>npm run build</code> & <code>npm start</code>	Production build + serve
<code>npx tsc --noEmit</code>	Type check
<code>npx eslint .</code>	Lint
<code>npm run prisma:generate</code>	Regenerate Prisma client
<code>npm run prisma:validate</code>	Validate schema
<code>npm run prisma:seed</code>	Destructive — wipes DB, reloads demo data + credentials

Env lives in `.env.local` (git-ignored): `DATABASE_URL`, `NEXTAUTH_SECRET`, `NEXTAUTH_URL`, `RESEND_API_KEY`, `RESEND_FROM_EMAIL`. APIs return 401/403 when permissions fail — check the server log (`next-server.log`) for Prisma errors and email warnings.

Sanity check pattern (like real QA): verified the permission matrix with live API calls — Director create/assign/committee-create all 403, reads 200; Deputy create/assign/committee 201, approve 200; notification routed to the Deputy on lead submit. That "two-layer test" (API + UI) is the pattern to repeat after any change.

13. Suggested learning path

1. **Read the data model** — `prisma/schema.prisma` (10 min). Everything else refers to these shapes.
2. **Read the module core** — `src/lib/qms-record-api.ts` + `components/qms.tsx` (the two "brains").
3. **Trace one module end to end** — open `app/api/capa/route.ts` → `module-route-factory.ts` → `app/api/documents/[id]/approve/route.ts` → `transitionModuleRecord`.
4. **Exercise permissions** — temporarily call an API with a different logged-in user (session cookie) and watch the 403s.
5. **Add a new field** — e.g. a `priority` on Documents: update schema → migrate → seed → `RecordForm` → `moduleConfig`. This touches every layer and teaches the architecture fast.
6. **Read the workflow factory** — `src/lib/module-route-factory.ts` is a compact example of "many routes, one implementation."

Tip: set breakpoints in `transitionModuleRecord` and step through an approve action in the dev server — you'll see permission check → DB write → history → notification in one function.